Cloud Workload Forecasting

Two-hour forecast of active VMs and aggregate CPU on the Azure VM trace

Your Name

October 28, 2026

1 Executive summary

This report describes a pipeline that forecasts the number of active VMs and the aggregate CPU utilisation of 10,000 Azure VMs for the final two hours of an 11.1-hour trace. The data was complete and regular, so the two targets were built directly on the native 5-minute grid. The first 110 timestamps were used for training and the last 24 were kept as the holdout.

The analysis showed that the workload is very stable at fleet level. No VM was created or deleted during the window, so the active VM count stayed at 10,000 and a last-value forecast predicted it exactly. Aggregate CPU stayed between 8.98% and 9.91%, although individual VMs ranged from idle to fully loaded. This meant that the forecasting problem was mainly about small movements around a level of about 9.4%, and that a simple baseline would be hard to beat.

Three baselines (persistence, a 6-step moving average and a seasonal-naive forecast) were compared with ridge regression and gradient-boosted trees, each in a recursive and a direct form. Models were compared with an expanding-window backtest inside the training data. Recursive ridge regression had the lowest validation error, 38% below persistence, and was selected. On the holdout, however, its error was almost twice that of persistence. All validation windows covered a period in which CPU was falling, while the holdout period was rising, so the trend-following models under-forecast it. The main lesson is that the validation design could not show this weakness, because the 110 training points contained only one kind of movement.

Based on these results, the recommended policy is to provision for the last observed CPU level plus a margin of 0.60 points, which is the 95th percentile of the largest two-hour rise seen in the training data. The learned models should only replace this baseline after they have been validated on a longer and more varied history.

2 Data understanding and preprocessing

The release contains 1,340,000 CPU readings from 10,000 VMs at 134 timestamps, spaced 300 seconds apart. There were no missing values, no duplicate VM–timestamp pairs and no gaps in the time grid, and every VM reported at every timestamp. As a result, no resampling or imputation was needed, and the native 5-minute cadence was kept. The pipeline still stops with an error if the spacing is broken, so that a gap in other data would not be filled silently.

The active VM count was calculated as the number of distinct VMs reporting at each timestamp, and aggregate CPU as the mean of their `avg_cpu` values. The full series was built first and then split chronologically into 110 training and 24 holdout timestamps. The holdout values were only read after `predictions.csv` had been written.

Some metadata fields could not be used as features. The fields `avg_cpu`, `max_cpu` and `p95_max_cpu` summarise each VM over its whole lifetime, which includes the holdout period, and VM deletion times are not known in advance in a real system. Using any of them would have leaked future information into the model, so they were excluded.

3 Workload analysis

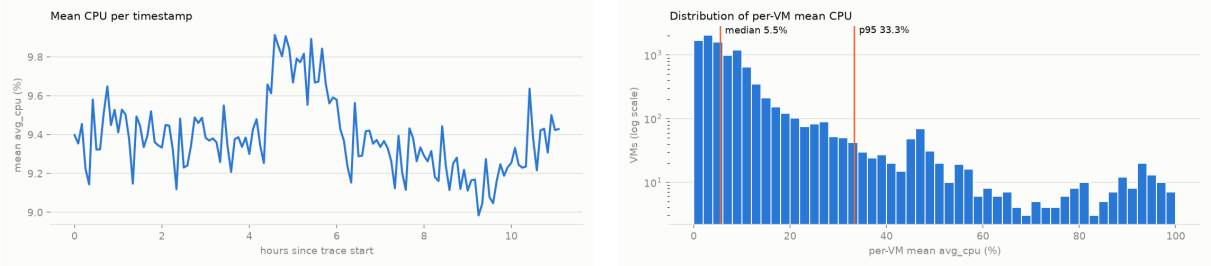


Figure 1: Left: aggregate CPU over the full trace. Right: distribution of VM-level CPU readings.

All VMs were created before the window began and none was deleted during it, with a median planned lifetime of about 720 hours. The VM count therefore has no dynamics in this dataset. This is a result of how the subset was selected rather than a property of real fleets, so the count forecast in this project cannot be considered tested.

CPU usage at VM level is strongly skewed. The median reading is 5.1% and the 95th percentile is 37%. Almost half of the VMs (46%) average below 5%, while only 2.2% average above 50%. The top 10% of VMs account for 44% of all CPU. This means that capacity risk depends on a small group of busy VMs, and that the fleet average hides most of the individual variation.

Aggregate CPU reached its highest values around 4.5 to 5.4 hours into the trace (9.91%) and its lowest around 9.3 hours (8.98%). The autocorrelation was 0.68 at one step, 0.53 at 30 minutes and only 0.13 at two hours. The series therefore depends strongly on its recent values but carries little information two hours ahead, which limits how much any model can gain over persistence at this horizon. The trace is also too short to contain a daily cycle, so calendar features and seasonal patterns were not expected to help.

VM categories differ in behaviour. Interactive VMs have a higher mean CPU (11.5%) and a larger spread (standard deviation 17.0) than delay-insensitive VMs (8.2% and 12.0). Since interactive workloads are also the most sensitive to slowdowns, they were later given more weight in the capacity recommendation.

Subscription and deployment IDs turned out to describe almost the same grouping (2,324 subscriptions against 2,359 deployments, normalised mutual information 0.999), so only subscriptions were analysed further. The top 1% of subscriptions account for 29% of all CPU, and the single most variable subscription explains about 11% of the variation in the aggregate. These few subscriptions are useful indicators for early warning of load increases.

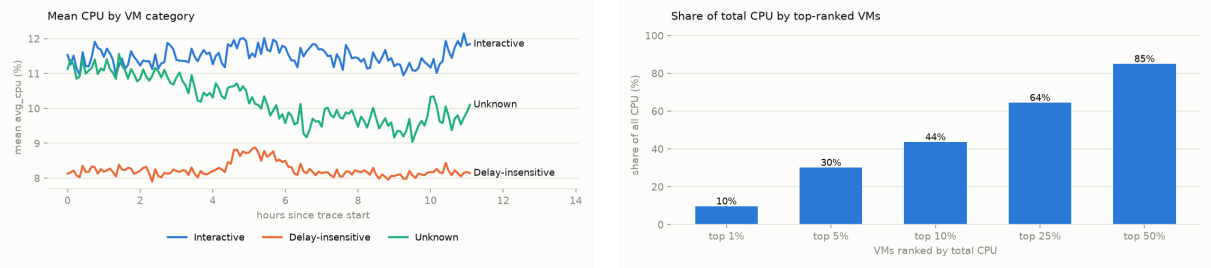


Figure 2: Left: CPU by VM segment. Right: share of total CPU held by the top-ranked VMs.

4 Forecasting approach

All features were computed at a forecast origin using only data available at that time. From the target series these were the current value, lags of 1, 2 and 5 steps, the last change, and the 6-step rolling mean and standard deviation. These were combined with fleet statistics from the same timestamp: the 95th-percentile VM CPU, the spread across VMs, the share of VMs above 50%, the mean of `max_cpu` and the

mean CPU per VM category. Calendar features were not used, for the reason given above.

Three baselines were implemented: persistence (the last value), a 6-step moving average and a seasonal-naive forecast with a two-hour period. Ridge regression was chosen as a simple linear model and gradient-boosted trees as a non-linear one. Each was used in two forms. The recursive form predicts one step ahead and feeds its own predictions back as inputs, while the direct form uses a single model with the forecast step (1 to 24) as an extra input. Both forms predict the change from the current value instead of the level, which keeps their forecasts close to the last observation.

The active VM count was forecast with persistence. A forecast based on VM lifetimes would have required deletion times, which are not available in practice, and the count did not change in the data anyway.

5 Validation methodology

An expanding-window backtest was used inside the 110 training rows. Each window was trained on all rows before its origin and then forecast the following 24 rows, which is the same task as the final forecast. The first origin had 62 rows of history and each later origin added 6 rows, giving five windows whose forecasts covered rows 62 to 109. The last window ends immediately before the holdout. All models and baselines were evaluated on the same windows, and the model with the lowest mean absolute error (MAE) was selected and refitted on all training data. This design reflects the production setting, in which a model is trained on all past data and asked to forecast the next two hours, but its five windows overlap and all come from the same part of the series.

6 Results versus baseline

Performance was measured with MAE and RMSE in CPU percentage points. To compare models with the baseline, each model's MAE was divided by the MAE of persistence, so values below 1 indicate an improvement. Since the denominator is the baseline error and not the CPU value, there is no problem with values close to zero.

	Validation (5 windows)			Holdout		
Model	MAE	RMSE	Rel. MAE	MAE	RMSE	Rel. MAE
`ridge_recursive` (selected)	0.118	0.145	0.62	0.290	0.349	1.98
`boosting_recursive`	0.124	0.165	0.66	0.176	0.210	1.20
`persistence`	0.189	0.241	1.00	0.147	0.185	1.00
`moving_average_6`	0.237	0.288	1.26	0.134	0.171	0.91
`seasonal_naive_24`	0.321	0.362	1.70	0.146	0.167	1.00
`boosting_direct`	0.329	0.396	1.74	0.217	0.259	1.48
`ridge_direct`	0.448	0.481	2.37	0.130	0.159	0.88

Table 1: Aggregate CPU errors. Rel. MAE is the model's MAE divided by the persistence MAE. The VM count forecast had an MAE of 0 on the holdout.

The ranking changed almost completely between validation and holdout. The two recursive models were better than all baselines in validation but worse than all of them on the holdout. Direct ridge regression achieved the best holdout result, 12% better than persistence, but it had been the worst model in validation and had over-forecast in every window. Its good holdout score therefore does not show that it is a better model, and choosing it after seeing the holdout would break the evaluation protocol. RMSE ranked the models in the same order as MAE, with RMSE/MAE between 1.1 and 1.3, which shows that the differences are not caused by a few large errors. The three baselines finished close to each other on the holdout. This suggests that, at this horizon and with this amount of data, the recent level of the series is the most reliable information available.

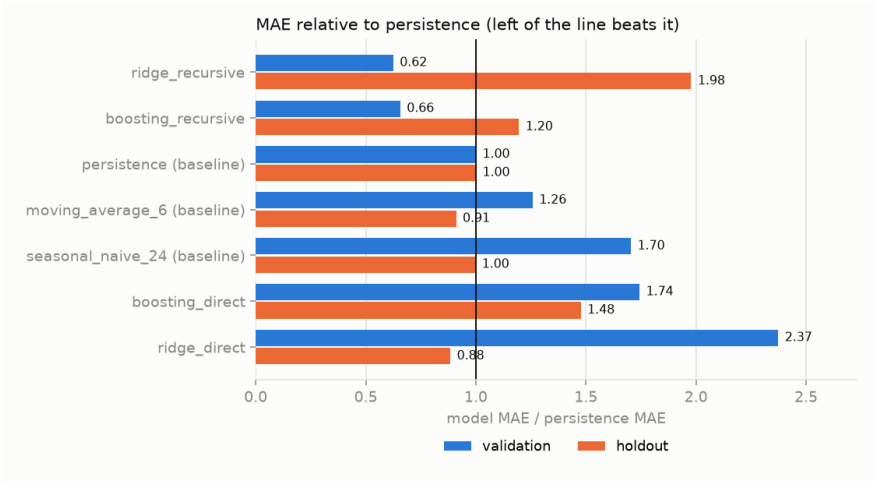


Figure 3: MAE relative to persistence in validation (blue) and on the holdout (orange).

7 Error and failure analysis

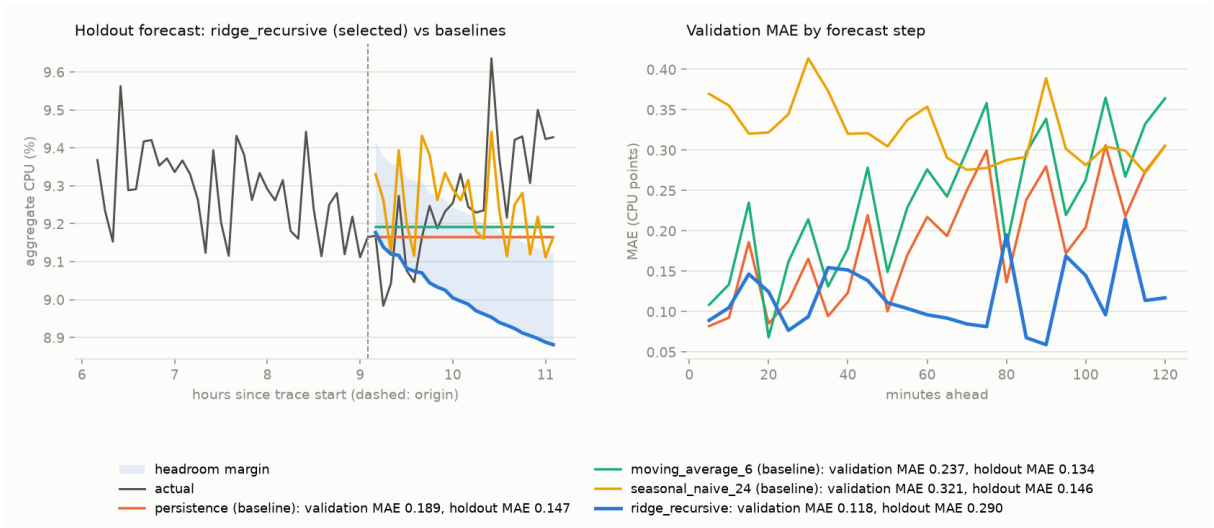


Figure 4: Left: holdout forecasts of the selected model and the baselines, with the headroom band. Right: validation MAE by forecast step.

The direction of the errors explains the change in ranking. In validation, persistence over-forecast by 0.16 points on average because CPU was falling, while the selected model was nearly unbiased but still under-forecast 72% of the steps. On the holdout, the selected model predicted a steady decline from 9.18% to 8.88%, whereas the actual value rose from 9.17% to 9.43%, giving an average under-forecast of 0.27 points. For capacity planning, under-forecasting is the more harmful type of error.

The cause is the period covered by validation. Every validation window started at a high level (9.33% to 9.84%) and ended lower (9.11% to 9.37%), so the models learned that a recent fall tends to continue. The holdout instead fell to 8.98% at 9.25 hours and then rose to 9.64% at 10.4 hours, where the selected model forecast 8.95%, an error of 0.68 points. The series tends to return to its mean of about 9.4%, but the models followed the recent trend. Because the five validation windows overlap and come from the same falling period, they did not give independent evidence, and this was the main weakness of the development process.

Errors also depend on the forecast step. The error of persistence grew from 0.08 at 5 minutes to 0.31 at two hours, while the recursive models stayed between 0.04 and 0.23 in validation. Their advantage was therefore real within the falling period but did not carry over to a different pattern.

Peak periods were defined as timestamps above the 90th percentile of the training data (9.67%). In validation, the recursive models performed worst on these steps, with MAE of 0.17 for ridge and 0.29 for boosting, compared with 0.08 for persistence. No holdout timestamp exceeded this threshold, so a holdout peak MAE could not be calculated, but the holdout maximum of 9.64% produced the selected model's largest error. The models are therefore least reliable when the fleet is busiest, which is when accurate forecasts matter most. Finally, the headroom margin of 0.238 points, set at the 95th percentile of the selected model's validation shortfall, covered only 42% of holdout steps instead of the intended 95%.

8 Capacity-planning recommendation

For the VM count, capacity should be planned on the current number of VMs, adjusted for any creations or deletions already scheduled by the platform. For CPU, the following policy is recommended.

1. Use the last observed aggregate CPU as the base forecast, since persistence was the most consistent method across both periods.
2. Add a safety margin of 0.60 points, the 95th percentile of the largest rise within any two-hour window of the training data. From the last training value of 9.17%, this provisions for 9.77%, about 6.5% above the current level.
3. Review capacity early if aggregate CPU exceeds the training 90th percentile (9.67%) or if one of the most variable subscriptions shows a sharp increase.
4. Reserve a larger part of the margin for interactive VMs, which are busier, more variable and more sensitive to delays than delay-insensitive workloads.

This margin rule was designed after the holdout had been scored and was not used to produce `predictions.csv`. When checked against the holdout, it covered all 24 steps, as the largest rise from the forecast origin was 0.47 points.

The policy accepts some over-provisioning to avoid under-provisioning. Too little capacity leads to CPU contention, throttling and higher latency, which affect interactive users directly and can break service agreements. Too much capacity only costs idle resources, and at fleet level an extra 0.6 points on an average load of 9.4% is a modest cost. Because the two errors have such different consequences, the margin is based on observed rises rather than on average error.

9 Limitations and future work

The main limitation is the length of the data. With 134 points covering about 11 hours, the trace contains no daily cycle and only one kind of movement in the training period, and the validation windows are strongly correlated. The VM count did not change, so its forecast was not tested. Model selection used MAE, which treats over- and under-forecasting equally even though their costs differ. A selection rule that excluded models under-forecasting more than half of the time would have chosen recursive boosting instead (holdout MAE 0.176).

Future work should include a feature that measures the distance from the running mean, so that models can learn the tendency to return to the average. With a longer trace, validation windows should cover both rising and falling periods. Quantile models could give prediction intervals directly, and separate forecasts for interactive and delay-insensitive VMs could support the segment-based part of the capacity policy.

Reproduction: `python -m cwfc.predict -data challenge_data -output predictions.csv` (see README).